

# Detecting Tomato Leaf Diseases by Image Processing Through Deep Convolutional Neural Networks

Md. Iqbal Hossain<sup>1</sup>, Sohely Jahan<sup>\*1</sup>, Md. Rashid Al Asif<sup>1</sup>, Md. Samsuddoha<sup>1</sup>, and Kawsar Ahmed<sup>†2,3</sup>

<sup>1</sup>Department of Computer Science and Engineering, University of Barishal, Barishal-8254, Bangladesh

<sup>2</sup>Group of Biophotonics, Department of Information and Communication Technology, Mawlana Bhashani Science and Technology University, Santosh, Tangail-1902, Bangladesh

<sup>3</sup>Health Informatics Research Lab, Department of Computer Science and Engineering, Daffodil International University, Daffodil Smart City, Birulia, Dhaka-1216, Bangladesh

Smart Agricultural Technology 5 (2023) 100301

## Abstract

Machine Learning (ML) and Deep Learning (DL) have already brought unprecedented success in the detection of various diseases of plant leaves, fruits, buds, flowers, and related agricultural components. In this paper, Deep Convolutional Neural Networks (DCNNs) are applied to tomato leaf disease detection. Gaussian and median filtering methods are combined with HSI and CMYK color models to evaluate the effect of image preprocessing on classification performance. The selected DCNN models are VGG-19, MobileNet-V2, and ResNet-50. The best results are obtained using Gaussian blur and Gaussian noise filters with RGB-to-CMYK color conversion: 98.27% for VGG-19, 94.98% for MobileNet-V2, and 99.53% for ResNet-50.

**Keywords:** Leaf disease detection; Deep learning; Neural networks; Color models; Median filter; Gaussian filter.

## 1 Introduction

The development of agriculture is closely related to the advancement of science, computer technology, and artificial intelligence (AI). Tomato (*Solanum lycopersicum* L.) is one of the most popular vegetable crops after potato. Tomato leaves are affected by several diseases such as Septoria leaf spot, early blight, and tomato yellow leaf curl virus. These diseases reduce production and may even destroy plants. Manual diagnosis is time-consuming and requires expert knowledge. Therefore, researchers are increasingly applying ML, DL, computer vision, and neural networks to agricultural disease diagnosis.

This work focuses on three common tomato leaf diseases: Septoria leaf spot, early blight, and yellow leaf curl virus. DCNNs are prioritized because they can automatically learn features, detect edges and low-level image patterns, and work well with transfer learning from large-scale datasets such as ImageNet. DCNNs are also suitable for disease images because disease symptoms may appear in different locations on different leaves.

The novelty of this work is the combination of filtering methods and color models to identify the preprocessing pipeline that gives the best classification performance. The paper applies Gaussian blur and Gaussian noise, median filtering, HSI conversion, and CMYK conversion before training and evaluating three DCNN models.

## 2 Related Work

AI was introduced by John McCarthy at the Dartmouth conference, and expert systems later became one of the earliest AI applications in agriculture. McKinion and Lemmon explored expert systems for agriculture, and COMAX was proposed for cotton crop management. Afterward, CNN, DBN, and other learning methods became common in agricultural image processing.

Kibriya et al. used VGG16 and GoogLeNet with machine-learning classifiers for tomato leaf disease detection. Jiang et al. proposed a ResNet-50 model and reported high test accuracy. Ashok et al. used CNN with Gaussian filtering, DWT, and GLCM features. Tm et al. compared AlexNet, GoogLeNet, and LeNet. Kumar et al. trained LeNet, VGG16, Xception, and ResNet50 on the PlantVillage dataset. These studies show that CNN-based models can perform strongly for tomato disease recognition.

## 3 Materials and Methodology

### 3.1 Data Source and Description

The dataset used in the study is the PlantVillage dataset from Kaggle. The selected classes are early blight, Septoria leaf spot, and yellow leaf curl virus. Around 7000 images are used for training and evaluation.

### 3.2 Overall Methodology

The overall workflow includes collecting the dataset, applying filters, splitting the dataset, resizing images, applying color models, generating augmented image data, building DCNN models, training, evaluating, and testing.

<sup>\*</sup>Corresponding author: sohely.cse@gmail.com

<sup>†</sup>Corresponding author: kawsar.ict@mbstu.ac.bd



Fig. 1. Images of our selected 3 leaf diseases of tomatoes.

model. The images were expanded further by Gaussian noise and Affine transformation which sums up a total of 6794 images. There is a lack of a relatively complete and high-quality crop disease database, the classification of some rare diseases or species in their paper.

Also in 2020, Asok et al. [10] proposed a CNN model to detect four types of tomato leaf conditions such as Healthy, Phoma Rot, Leaf Miner, and Target Spot. In the preprocessing steps, they applied the Gaussian filter to reduce the noise of the images. DWT (Discrete Wavelet Transform) and GLCM (Gray Level Co-occurrence Matrix) are applied in fea-

Figure 1: Images of the selected three tomato leaf diseases.

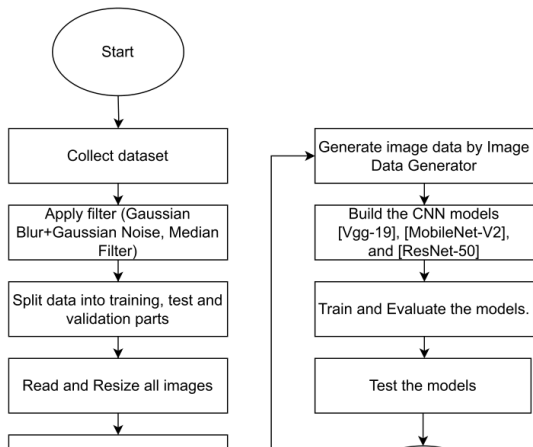


Figure 2: Overall methodology of the research.

### 3.3 Filtering in Pre-processing Steps

The study applies Gaussian blur, Gaussian noise, and median filtering separately to compare their effects on the dataset.

#### 3.3.1 Gaussian Blur

Gaussian blur is widely used to reduce noise and details in images. It is expressed as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}, \quad (1)$$

where  $x$  and  $y$  are pixel coordinates,  $\sigma$  is the standard deviation, and  $G(x, y)$  is the Gaussian kernel.

#### 3.3.2 Gaussian Noise

Gaussian noise is generated from a Gaussian distribution and may simulate camera or natural noise:

$$I_{noisy}(x, y) = I(x, y) + \epsilon(x, y), \quad (2)$$

where  $I(x, y)$  is the original image and  $\epsilon(x, y)$  is Gaussian noise.

#### 3.3.3 Median Filter

Median filtering replaces a pixel value with the median value of surrounding pixels:

$$I_{out}(x, y) = \text{median}(I_{in}(x-k : x+k, y-k : y+k)), \quad (3)$$

where  $k$  is the radius of the median filter kernel.

### 3.4 Data Split

The images are randomly divided into training, validation, and test sets. The split used is 70% training, 20% validation, and 10% testing.

### 3.5 Color Models

Two color models are applied: HSI and CMYK.

#### 3.5.1 HSI Color Model

The HSI model is commonly used in image processing for enhancement, contrast adjustment, and segmentation. The formulas are:

$$H = \cos^{-1} \left( \frac{0.5((R - G) + (R - B))}{\sqrt{(R - G)^2 + (R - B)(G - B)}} \right), \quad (4)$$

$$S = 1 - \frac{3 \min(R, G, B)}{R + G + B}, \quad (5)$$

$$I = \frac{R + G + B}{3}. \quad (6)$$

### 3.5.2 CMYK Color Model

The CMYK model is subtractive, unlike RGB. RGB values are converted as:

$$CMYK = (C', M', Y', K), \quad (7)$$

where

$$C' = \frac{C - K}{1 - K + 10^{-8}}, \quad M' = \frac{M - K}{1 - K + 10^{-8}}, \quad (8)$$

$$Y' = \frac{Y - K}{1 - K + 10^{-8}}, \quad K = \min(C, M, Y), \quad (9)$$

with  $C = 1 - R$ ,  $M = 1 - G$ , and  $Y = 1 - B$ .

### 3.6 Image Data Generation

ImageDataGenerator is used for augmentation. Rotation, zooming, shifting, horizontal flipping, and shear range are applied so that the model can learn more robust features.

### 3.7 DCNN Architectures

Three DCNN models are constructed: VGG-19, MobileNet-V2, and ResNet-50.

### 3.8 Activation Functions, Optimizer, and Loss

ReLU is applied in the fully connected layers:

$$f(x) = \max(0, x). \quad (10)$$

Softmax is used for multi-class classification:

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}. \quad (11)$$

The Adam optimizer is used because it performs well in deep networks. The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (12)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (13)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (14)$$

The sparse categorical cross-entropy loss is:

$$L(y_{\text{true}}, y_{\text{pred}}) = - \sum_{i=1}^C y_{\text{true},i} \log(y_{\text{pred},i}). \quad (15)$$

Early stopping is used to reduce overfitting.

### 3.9 Confusion Matrix and Metrics

The evaluation metrics are accuracy, precision, recall, and F1-score:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (16)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (17)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (18)$$

$$F1 = \frac{2(\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}. \quad (19)$$

## 4 Results and Performance Discussion

The results are discussed under four preprocessing combinations:

1. Gaussian blur and Gaussian noise filter with RGB-to-CMYK conversion.
2. Gaussian blur and Gaussian noise filter with RGB-to-HSI conversion.
3. Median filter with RGB-to-CMYK conversion.
4. Median filter with RGB-to-HSI conversion.

### 4.1 Gaussian Blur and Gaussian Noise with RGB-to-CMYK

For VGG-19, the training accuracy is 100%, validation accuracy is 98.07%, and final test accuracy is 98.27%. For MobileNet-V2, the training and validation accuracies are 98.58% and 95.02%, and final accuracy is 94.98%. ResNet-50 gives 100% training accuracy, 99.28% validation accuracy, and 99.53% test accuracy.

### 4.2 Gaussian Blur and Gaussian Noise with RGB-to-HSI

VGG-19 achieves 98.12% final accuracy. MobileNet-V2 achieves 95.13% final accuracy. ResNet-50 achieves 99.37% final accuracy.

### 4.3 Median Filter with RGB-to-CMYK

For VGG-19, the final accuracy is 98.16%. For MobileNet-V2, the final accuracy is 94.93%. For ResNet-50, the final accuracy is 98.92%.

### 4.4 Median Filter with RGB-to-HSI

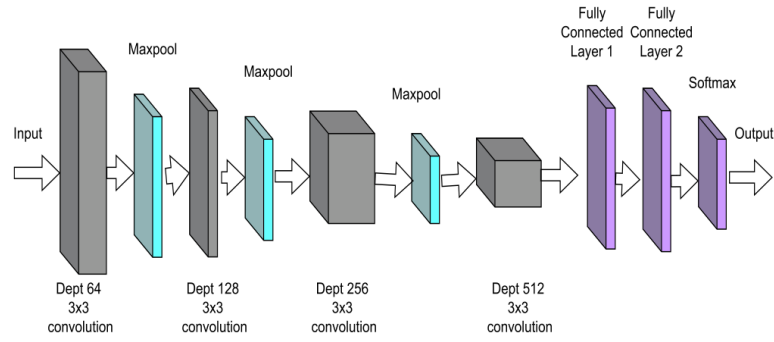
VGG-19 achieves 95.08% final accuracy, MobileNet-V2 achieves 94.93%, and ResNet-50 achieves 99.23%.

### 4.5 Choosing the Best Model

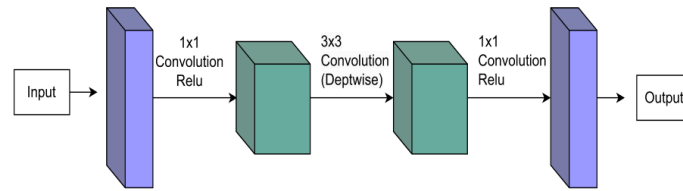
Table 1 shows that every model achieves more than 94% accuracy. ResNet-50 performs best overall, especially with Gaussian blur and Gaussian noise filtering combined with the CMYK color model.

## 5 Conclusion

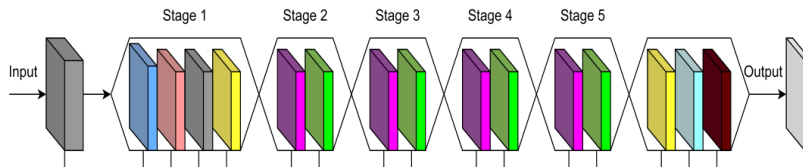
Computer and information technology continue to improve agriculture. In many developing regions, manual diagnosis of leaf disease is still common, but this method is slow and complex. This study shows that tomato leaf diseases can be detected effectively using image preprocessing and DCNN models. Among the tested combinations, Gaussian blur and Gaussian noise filtering with RGB-to-CMYK conversion provides the best overall performance. ResNet-50 achieves the highest accuracy of 99.53%. Future work may



(a) Architecture of VGG-19 model



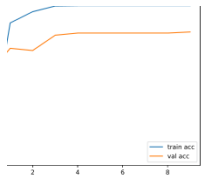
(a) Architecture of MobileNet-V2 model

**Figure 3:** Architecture of the proposed DCNN models: VGG-19, MobileNet-V2, and ResNet-50.**Table 1:** Overall performance table.

| Method                                     | Model        | Accuracy |
|--------------------------------------------|--------------|----------|
| Gaussian blur and Gaussian noise with CMYK | VGG-19       | 98.27%   |
|                                            | MobileNet-V2 | 94.98%   |
|                                            | ResNet-50    | 99.53%   |
| Gaussian blur and Gaussian noise with HSI  | VGG-19       | 98.12%   |
|                                            | MobileNet-V2 | 95.13%   |
|                                            | ResNet-50    | 99.37%   |
| Median filter with CMYK                    | VGG-19       | 98.16%   |
|                                            | MobileNet-V2 | 94.93%   |
|                                            | ResNet-50    | 98.92%   |
| Median filter with HSI                     | VGG-19       | 95.08%   |
|                                            | MobileNet-V2 | 94.93%   |
|                                            | ResNet-50    | 99.23%   |

**Table 2:** Comparison with selected existing works.

| Reference                 | Proposed model and accuracy                             |
|---------------------------|---------------------------------------------------------|
| Jiang et al.              | ResNet-50: 98%                                          |
| Kibriya et al.            | GoogLeNet: 99.23%; VGG16: 98%                           |
| Brahimi et al.            | CNN: 99.18%                                             |
| Ashok et al.              | CNN: 98.12%                                             |
| Kumar et al.              | VGG16: 99.25%; LeNet, ResNet50, Xception: 91%–98.65%    |
| Proposed best methodology | VGG-19: 98.27%; MobileNet-V2: 94.98%; ResNet-50: 99.53% |



ining and validation ac-  
s

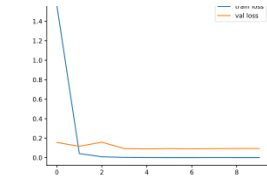
|           | precision | recall | f1-score | support |
|-----------|-----------|--------|----------|---------|
| 0         | 0.95      | 0.94   | 0.94     | 80      |
| 1         | 0.97      | 0.97   | 0.97     | 142     |
| 2         | 1.00      | 1.00   | 1.00     | 415     |
| accuracy  |           |        | 0.98     | 637     |
| acro avg  | 0.97      | 0.97   | 0.97     | 637     |
| ghted avg | 0.98      | 0.98   | 0.98     | 637     |

racy: 0.9827

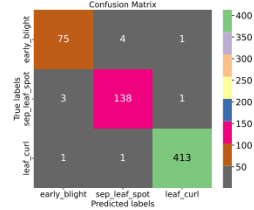
Classification report

aussian Blur and the Gaussian Noise filter with RGB to CMYK color  
n in VGG-19.

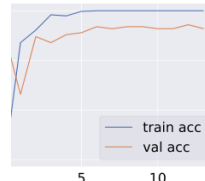
**Figure 4:** Gaussian blur and Gaussian noise with RGB-to-CMYK  
conversion in VGG-19.



(b) Training and validation  
losses



(d) Confusion matrix

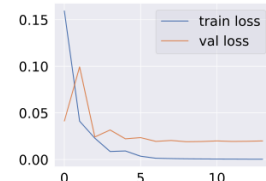


aining and Validation ac-  
ies

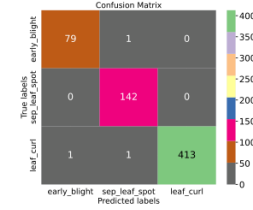
|       | precision | recall | f1-score | support |
|-------|-----------|--------|----------|---------|
| 0     | 0.99      | 0.99   | 0.99     | 80      |
| 1     | 0.99      | 1.00   | 0.99     | 142     |
| 2     | 1.00      | 1.00   | 1.00     | 415     |
| avg   | 0.99      | 0.99   | 0.99     | 637     |
| d avg | 1.00      | 1.00   | 1.00     | 637     |

r: 0.9953

) Classification report

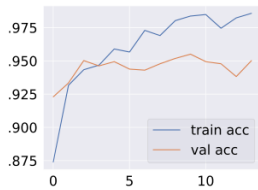


(b) Training and validation  
losses



(d) Confusion matrix

**Figure 6:** Gaussian blur and Gaussian noise with RGB-to-CMYK  
conversion in ResNet-50.

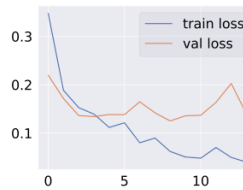


(a) Training and Validation ac-  
curacies

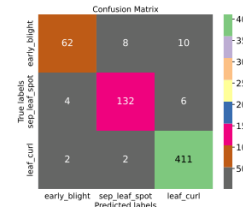
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.91      | 0.78   | 0.84     | 80      |
| 1            | 0.93      | 0.93   | 0.93     | 142     |
| 2            | 0.96      | 0.99   | 0.98     | 415     |
| accuracy     |           |        | 0.95     | 637     |
| macro avg    | 0.93      | 0.90   | 0.91     | 637     |
| weighted avg | 0.95      | 0.95   | 0.95     | 637     |

Accuracy: 0.9498

(c) Classification report

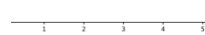


(b) Training and Validat  
losses



(d) Confusion matrix

**Figure 5:** Gaussian blur and Gaussian noise with RGB-to-CMYK  
conversion in MobileNet-V2.

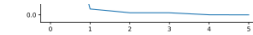


aining and validation ac-  
y

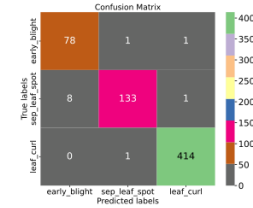
|     | precision | recall | f1-score | support |
|-----|-----------|--------|----------|---------|
| 0   | 0.91      | 0.97   | 0.94     | 80      |
| 1   | 0.99      | 0.94   | 0.96     | 142     |
| 2   | 1.00      | 1.00   | 1.00     | 415     |
| cy  |           |        | 0.98     | 637     |
| avg | 0.96      | 0.97   | 0.97     | 637     |
| avg | 0.98      | 0.98   | 0.98     | 637     |

: 0.9812

) Classification report



(b) Training and validation  
losses



(d) Confusion matrix

Gaussian Blur and the Gaussian Noise filter with RGB to HSI color con-  
in VGG-19.

-trained and proficient in predicting the outcomes. Furthermore,  
c) presents the classification report, and the corresponding con-  
matrix, for the true and predicted levels, is depicted in Fig. 7(d).

**Figure 7:** Gaussian blur and Gaussian noise with RGB-to-HSI  
conversion in VGG-19.

Figure 8: Gaussian blur and Gaussian noise with RGB-to-HSI conversion in MobileNet-V2.

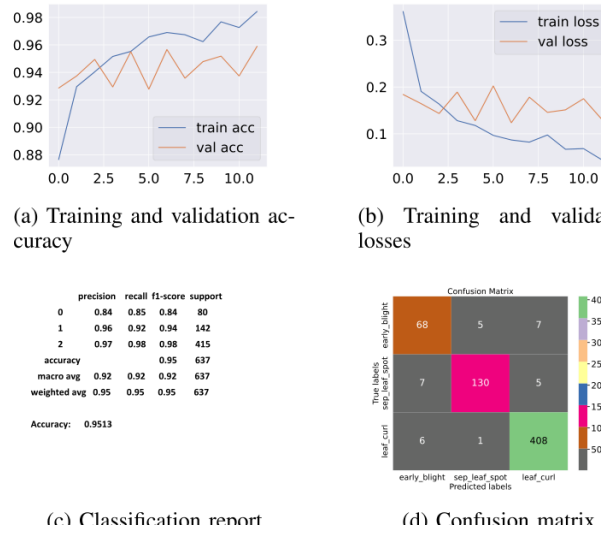


Figure 8: Gaussian blur and Gaussian noise with RGB-to-HSI conversion in MobileNet-V2.

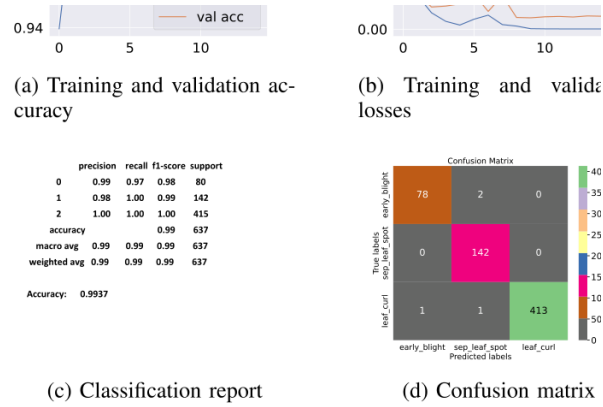


Fig. 9. Gaussian Blur and the Gaussian Noise filter with RGB to HSI color version in ResNet-50.

in Fig. 9(a)). Fig. 9(b) displays the training and validation losses training loss in this manner is zero and the validation loss is 1.53%

Figure 9: Gaussian blur and Gaussian noise with RGB-to-HSI conversion in ResNet-50.

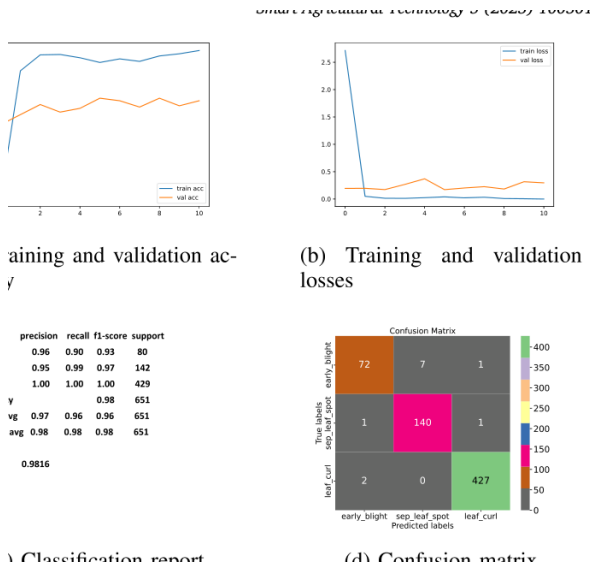
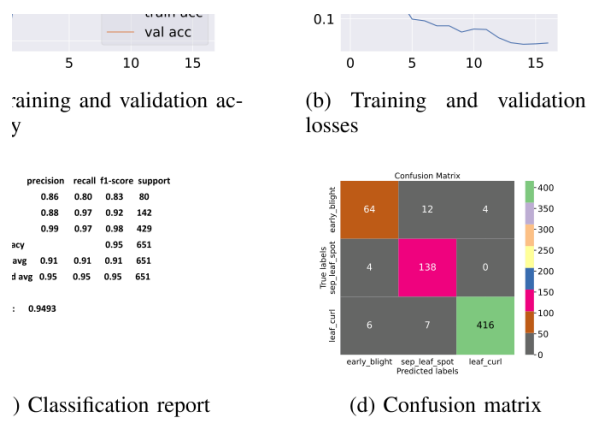


Figure 10: Median filter with RGB-to-CMYK conversion in VGG-19.



1. Median filter with RGB to CMYK color conversion in MobileNet-V2.

### MobileNet-V2

MobileNet-V2, the training accuracy and the validation accuracy

Figure 11: Median filter with RGB-to-CMYK conversion in MobileNet-V2.

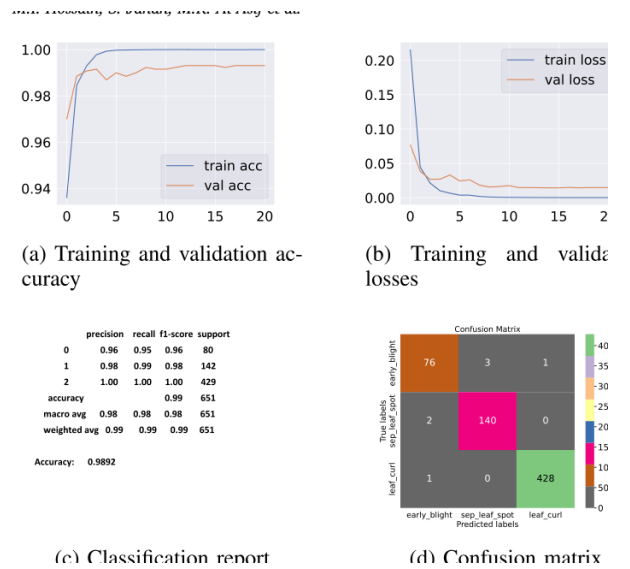


Figure 12: Median filter with RGB-to-CMYK conversion in ResNet-50.

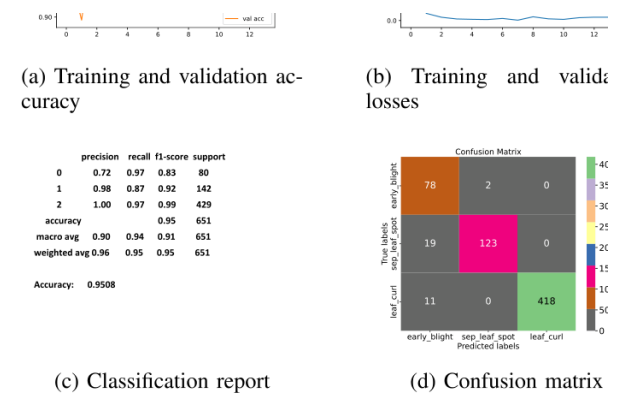
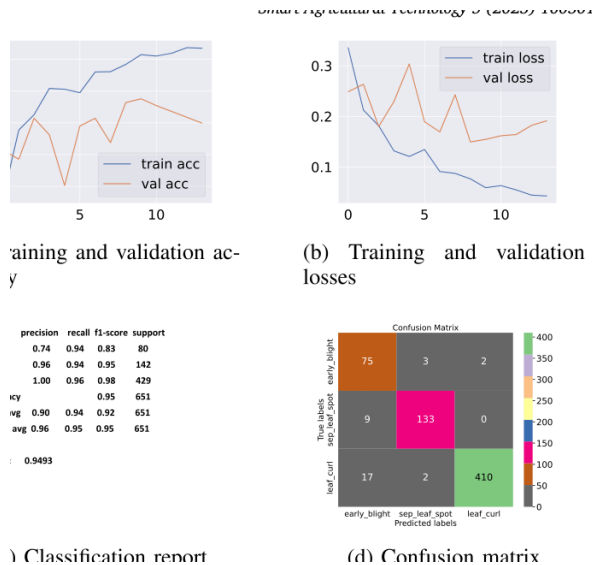


Fig. 13. Median filter with RGB to HSI color conversion in VGG-19.

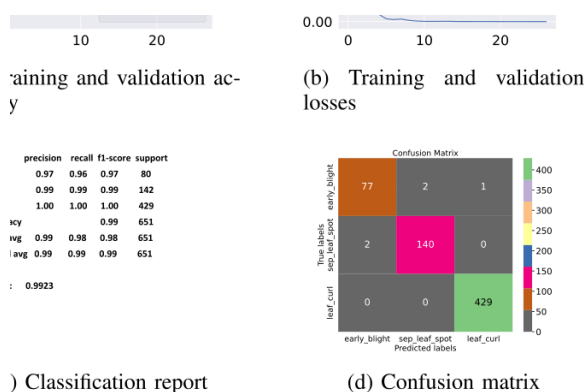
### 3.4. Median filter with RGB to HSI color conversion

#### 3.4.1. VGG-19

Figure 13: Median filter with RGB-to-HSI conversion in VGG-19.



**Figure 14:** Median filter with RGB-to-HSI conversion in MobileNet-V2.



**Figure 15:** Median filter with RGB to HSI color conversion in ResNet-50.

#### ResNet-50

Figure 15(a) demonstrated 100% training accuracy and 99.54% validation accuracy. Fig. 15(b) displays that the training loss also is zero.

**Figure 15:** Median filter with RGB-to-HSI conversion in ResNet-50.

use larger datasets, real field images, and hybrid combinations of CNNs with classical machine-learning models such as decision trees and SVM.

## Declaration of Competing Interest

The authors declare no known competing financial interests or personal relationships that could have influenced the work.

## Data Availability

Data will be made available on request.

## Funding

This research received no external funding.

## References

- [1] National Plant Data Center, NRCS, USDA, "The scientific name of Tomato," accessed 13 March 2023.
- [2] National Plant Data Center, NRCS, USDA, "The scientific name of Potato," accessed 13 March 2023.
- [3] J. McCarthy, M. L. Minsky, N. Rochester, and C. E. Shannon, "A proposal for the Dartmouth summer research project on artificial intelligence," *AI Magazine*, vol. 27, no. 4, p. 12, 2006.
- [4] K. Jha, A. Doshi, P. Patel, and M. Shah, "A comprehensive review on automation in agriculture using artificial intelligence," *Artificial Intelligence in Agriculture*, vol. 2, pp. 1–12, 2019.
- [5] J. M. McKinion and H. E. Lemmon, "Expert systems for agriculture," *Computers and Electronics in Agriculture*, vol. 1, no. 1, pp. 31–40, 1985.
- [6] H. Lemmon, "COMAX: An expert system for cotton crop management," *Science*, vol. 233, no. 4759, pp. 29–33, 1986.
- [7] D. I. Patricio and R. Rieder, "Computer vision and artificial intelligence in precision agriculture for grain crops: a systematic review," *Computers and Electronics in Agriculture*, vol. 153, pp. 69–81, 2018.
- [8] H. Kibriya, R. Rafique, W. Ahmad, and S. M. Adnan, "Tomato leaf disease detection using convolution neural network," in *IBCAST*, 2021, pp. 346–351.
- [9] D. Jiang, F. Li, Y. Yang, and S. Yu, "A tomato leaf diseases classification method based on deep learning," in *CCDC*, IEEE, 2020, pp. 1446–1450.
- [10] S. Ashok et al., "Tomato leaf disease detection using deep learning techniques," in *ICCES*, 2020, pp. 979–983.

- [11] P. Tm et al., “Tomato leaf disease detection using convolutional neural networks,” in *IC3*, 2018, pp. 1–5.
- [12] A. Kumar and M. Vani, “Image based tomato leaf disease detection,” in *ICCCNT*, IEEE, 2019, pp. 1–6.
- [13] Emmarex, “Dataset of diseased plant leaf images and corresponding labels,” Kaggle, accessed 20 December 2022.
- [14] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [15] Hasty.ai, “Gaussian Noise,” accessed 17 March 2023.
- [16] B. Jahne, *Digital Image Processing*. Springer, 2005.
- [17] R. W. G. Hunt, *The Reproduction of Colour*, 6th ed. Wiley, 2004.
- [18] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” arXiv:1409.1556, 2014.
- [19] M. Sandler et al., “MobileNetV2: Inverted residuals and linear bottlenecks,” in *CVPR*, 2018, pp. 4510–4520.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *CVPR*, 2016, pp. 770–778.
- [21] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in *ICML*, 2010, pp. 807–814.
- [22] J. S. Bridle, “Probabilistic interpretation of feedforward classification network outputs,” *Neurocomputing*, vol. 1, no. 3, pp. 145–163, 1990.
- [23] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv:1412.6980, 2014.
- [24] F. Chollet and L. Keras, “A concise API for deep learning,” arXiv:1910.00547, 2019.
- [25] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [26] L. Prechelt, “Early stopping - but when?” 2012, pp. 53–67.
- [27] D. M. W. Powers, “Evaluation: from precision, recall and F-factor to ROC,” *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.
- [28] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in *ICML*, ACM, 2006, pp. 233–240.
- [29] C. J. Van Rijsbergen, *Information Retrieval*, 2nd ed. Butterworths, 1979.
- [30] M. Brahimi, K. Boukhalfa, and A. Moussaoui, “Deep learning for tomato diseases: classification and symptoms visualization,” *Applied Artificial Intelligence*, vol. 31, no. 4, pp. 299–315, 2017.